

Exercice de synthèse: jeu du pendu

Cet exercice vise à écrire un programme un peu plus conséquent, qui demandera la définition et l'utilisation de fonctions, des exécutions conditionnelles (*if*) et des boucles.

Le but de l'exercice est de programmer un jeu du pendu (voir par exemple <https://www.regles-jeux-plein-air.com/regle-du-pendu/>). Nous allons progressivement construire des fonctions utiles pour résoudre différents aspects du problème – certaines de ces fonctions s'appuieront sur d'autres fonctions définies précédemment. Ensuite, nous écrirons le cœur du programme, qui utilisera à son tour les différentes fonctions.

Quelques recommandations générales :

- Chaque fonction, y compris la fonction finale qui implémentera l'ensemble du jeu, doit être assez courte (15-20 lignes maximum, sauf cas exceptionnel) : c'est là une bonne pratique générale en programmation qui aide grandement la lisibilité du code.
- Veillez à la précision : par exemple, une fonction qui renvoie un résultat n'est pas la même chose qu'une fonction qui affiche ce résultat. Des différences en apparence minimales entre l'énoncé et votre code peuvent avoir de grands impacts.
- Réfléchissez également à tous les cas de figure possible : si une fonction est censée mettre un mot en majuscules, que se passe-t-il si le mot reçu ne comporte pas que des lettres, ou s'il est vide ? Il n'est pas forcément nécessaire de traiter explicitement chacun de ces cas (on peut par exemple préciser dans la description de la fonction que tel cas est interdit), mais il faut se poser la question de toutes les situations qui peuvent se poser.
- Testez votre code au fur et à mesure : à chaque fois qu'une fonction est écrite, prenez le temps de la tester en l'exécutant sur quelques données connues, y compris les cas spéciaux discutés ci-dessus, et de vérifier qu'elle a bien le comportement attendu. Certaines approches modernes, telles que le « *Test Driven Development* » (*TDD*) recommandent même d'écrire la série de tests *avant* de coder la fonction elle-même.

Avant toute chose, créez un projet JeuPendu, dans lequel vous écrirez les fonctions ci-dessous.

Partie 1 (connaissances préalables : fonctions, exécution conditionnelle)

Pour simplifier les choses, notre jeu n'utilisera que des lettres majuscules non accentuées. Nous allons donc refuser toute saisie clavier qui ne soit pas une lettre non accentuée et convertir en majuscules tout ce qui est saisi. Les fonctions suivantes nous seront utiles.

1. **Ecrire une fonction `EstUneLettre()`. Cette fonction reçoit un caractère (type `char`) et renvoie un booléen indiquant si le caractère reçu est bien une lettre (majuscule ou minuscule) non accentuée.**

Indices :

- Dans la norme Unicode, les lettres de l'alphabet latin ont des codes consécutifs : les lettres 'A' à 'Z' correspondent aux valeurs 65 à 90 et les lettres 'a' à 'z' aux valeurs 97 à 122.
- Mais en fait, il n'est même pas nécessaire de connaître les valeurs ci-dessus : si `Lettre` est une variable de type `char`, on peut parfaitement écrire une comparaison comme
`if (Lettre > 'A') ...`

2. **Ecrire une fonction `LettreEnMajuscules()`. Cette fonction reçoit un caractère. Si ce caractère correspond à une lettre minuscule, la fonction renvoie la lettre majuscule correspondante. Sinon, elle renvoie le caractère reçu sans le modifier.**

Indices :

- La fonction doit toujours renvoyer quelque chose, même si le caractère reçu n'est pas une lettre minuscule.
- Si c'est une lettre minuscule, il faudra obtenir le « numéro » de la lettre dans l'alphabet. Il est possible de le faire avec une opération mathématique impliquant cette lettre et le caractère 'a'.
- Il faudra ensuite appliquer un décalage correspondant à ce numéro d'ordre, mais à partir du caractère 'A'.
- Attention aux problèmes de casting : en C#, même si les variables `Lettre1` et `Lettre2` sont de type `char`, l'opération :
`Lettre1 + Lettre2`
convertit automatiquement le résultat en `int`. Il faudra donc forcer une conversion via une instruction comme :
`NouvelleLettre = (char) (Lettre1 + Lettre2) ;`

Partie 2 (connaissances préalables : boucles)

Passons maintenant à la fonction qui servira à saisir les lettres proposées par le joueur essayant de deviner le mot.

3. Ecrire une fonction `SaisirLettre()` qui

- a. Affiche un message invitant l'utilisateur à taper une lettre non accentuée, suivie de `<enter>`.
- b. Lit ce que l'utilisateur tape au clavier.
- c. Vérifie si la saisie correspond à ce qui est attendu. Si c'est bien le cas, la fonction renvoie cette lettre (type `char` donc) convertie en majuscules si nécessaire. Si l'utilisateur a tapé autre chose, la fonction répète le message, attend une nouvelle saisie au clavier, et ainsi de suite jusqu'à ce qu'une lettre non accentuée soit tapée. La fonction ne s'arrêtera donc que lorsqu'une lettre non accentuée aura bien été entrée.

Indices :

- Il ne faut pas réinventer la roue ! Les fonctions écrites lors des étapes précédentes peuvent évidemment être appelées : elles résolvent une bonne partie du problème.
- Attention à prévoir tous les cas : l'utilisateur pourrait saisir un chiffre, un mot, ou rien du tout. La fonction doit marcher dans toutes ces situations.
- Si `Texte` est une variable de type `string`, on peut obtenir sa longueur via `Texte.Length`.
- On peut lire les différents caractères d'une chaîne comme pour un tableau : `Texte[0]` est par exemple le premier caractère de la chaîne.

La prochaine étape est de saisir le mot à faire deviner. Lui aussi devra être composé uniquement de lettres non accentuées, et sera converti en majuscules.

4. Ecrire une fonction `EstComposeDeLettres()` qui reçoit en paramètre une chaîne de caractères et renvoie `true` si cette chaîne correspond à un mot composé uniquement de lettres non accentuées (et que ce mot comporte au moins une lettre) et `false` dans tous les autres cas.

5. Ecrire une fonction `MotEnMajuscules()` qui reçoit une chaîne de caractères correspondant à un mot et renvoie ce mot converti en majuscules. Si des lettres sont déjà en majuscules, ou si des caractères ne sont pas des lettres, ils sont inchangés.

Indices :

- `C#` ne permet pas d'écrire dans une chaîne de caractères : une instruction comme `Mot[0] = 'A' ;` n'est pas autorisée. A la place, il suffit de construire le nouveau mot à partir d'une chaîne vide (`""`) à laquelle on concatène les caractères via l'opérateur `+`.
- A part cela, il y a en fait très peu à faire pour cette fonction : la plupart des cas particuliers sont déjà traités par les fonctions précédentes, il suffit de les utiliser correctement.

6. **Ecrire une fonction `SaisirMot()` qui invite l'utilisateur à taper un mot composé de lettres non accentuées et renvoie ce mot mis en majuscules. Comme pour `SaisirLettre()`, la fonction ne termine pas avant que l'utilisateur n'ait saisi un mot conforme.**

Passons aux fonctions qui nous permettront de tester les propositions du joueur et d'afficher les lettres correctes à leur place dans le mot à trouver. Les lettres non encore trouvées seront représentées par des points.

7. **Ecrire une fonction `InitialiserMotVide()` qui reçoit un mot de référence et renvoie une chaîne de la même longueur, composée uniquement de points. Par exemple, `InitialiserMotVide("MONTRE")` doit renvoyer la chaîne `"....."`.**
8. **Ecrire une fonction `EstDansMot()`, qui reçoit un caractère ainsi qu'une chaîne de caractères et renvoie un booléen indiquant si le caractère apparaît au moins une fois dans la chaîne.**
9. **Ecrire une fonction `CompleterMot()`. Cette fonction :**
 - a. **Reçoit 2 chaînes de caractères, représentant le mot à compléter et le mot de référence, ainsi qu'un caractère correspondant à la lettre proposée.**
 - b. **Renvoie une chaîne de caractères correspondant au mot à compléter dans lequel ont été ajoutées toutes les occurrences de la lettre proposée, par rapport au mot de référence.**

Exemple : `CompleterMot("..IM..", "ANIMAL", 'A')` doit renvoyer `"A.IMA."`.

Partie 3 : première mise en œuvre du jeu

Nous avons maintenant suffisamment d'éléments pour mettre en œuvre une première version du jeu. Cette version n'affichera pas le dessin du pendu, mais un simple message « vous avez fait ... erreurs ». Mais nous allons néanmoins écrire une fonction `AfficherPendu()` qui affiche ce message. Par la suite, quand nous voudrons que le programme fasse un véritable dessin, il nous suffira de modifier cette fonction, sans devoir toucher au reste du programme.

10. **Ecrire une fonction `AfficherPendu()` qui reçoit un entier indiquant le nombre d'erreurs commises et affiche un message indiquant « Vous avez fait ... erreurs ».**

Venons-en à la fonction `JeuDuPend()`, qui mettra en œuvre le jeu complet (en utilisant les fonctions définies précédemment). Cette fonction est plus complexe que les précédentes (mais pas beaucoup plus longue : à titre de comparaison, la fonction que j'ai écrite fait une trentaine de lignes), mais mérite d'y consacrer du temps.

11. Ecrire une fonction `JeuDuPend()` qui met en œuvre l'ensemble du jeu, depuis la saisie du mot à faire deviner jusqu'à la victoire ou la défaite du joueur. Pour fixer les idées, disons que le joueur perd le jeu s'il propose 6 lettres incorrectes.

Commencez par essayer de l'écrire vous-mêmes : réfléchissez au problème, aux différentes étapes, à ce qu'il faut répéter (boucles) et aux conditions d'arrêt, etc. Il ne faut pas forcément commencer immédiatement à écrire du code : cela vaut la peine de réfléchir « sur papier » et de noter les grandes lignes de la résolution avant de se mettre à coder. Dites-vous bien qu'il n'y a aucun mal à ne pas réussir du premier coup ou à découvrir en cours de route que vous avez oublié certains aspects.

Si vous ne parvenez pas à écrire cette fonction, lisez les indications ci-dessous (pas forcément d'une traite : n'hésitez pas à revenir à votre code dès que vous pensez en avoir lu assez pour avancer) :

- Nous aurons au moins besoin de deux chaînes de caractères¹ : une qui contient le mot à deviner et une autre qui contient le mot partiellement trouvé (sous une forme comme "A.I.M.A."). Nous aurons aussi besoin d'une variable entière pour compter le nombre d'erreurs, que nous initialisons donc à 0.
- La première étape est de saisir le mot à faire deviner, puis de le mettre en majuscules. Nous pouvons le faire immédiatement avec les fonctions ci-dessus. Il faut ensuite initialiser le mot partiellement trouvé en une chaîne contenant autant de points qu'il y a de lettres à trouver. Ici encore, nous avons écrit une fonction qui fait exactement cela.
- Le cœur de la fonction est une boucle qui continuera jusqu'à la fin du jeu. Cette boucle peut s'arrêter dans deux cas : soit parce que le joueur a trouvé le mot (gagné) soit parce qu'il a fait le nombre maximum d'erreurs (perdu). Nous utiliserons deux variables booléennes `Gagne` et `Perdu` : toutes deux sont fausses au départ, et la boucle s'arrête dès que l'une devient vraie.
- La boucle doit effectuer les opérations suivantes (notons que, pour la plupart des étapes, nous avons écrit une fonction faisant exactement ce qu'il faut : il y a donc très peu de code à écrire) :
 - Lire au clavier la proposition du joueur, en s'assurant qu'il tape une seule lettre non accentuée, puis la convertir en majuscules.
 - Vérifier si cette lettre est dans le mot à trouver.
 - Si oui, compléter le mot à trouver en remplaçant les points par la lettre proposée aux endroits appropriés. Ensuite, il ne faut pas oublier de vérifier si le mot est complet (par exemple en le comparant au mot à trouver) : si oui, le joueur a gagné.

¹ Nous pourrions évidemment aussi utiliser des tableaux, mais c'est une matière que nous ne couvrirons que plus tard dans ce cours, et, à la base, une chaîne de caractères n'est rien d'autre qu'un tableau de lettres.

- Si non, il faut incrémenter le compteur d'erreurs, puis vérifier si on a atteint le nombre maximum d'erreurs autorisées (6) : si oui, le joueur a perdu.
- Afficher un résumé de la situation : le mot partiellement trouvé, le nombre d'erreurs... Pour la lisibilité du code, cela vaut la peine d'en faire une fonction séparée, `AfficherSituation()`, en lui passant les paramètres nécessaires.
- Une fois la boucle terminée, il n'y a plus qu'à tester si on termine sur une victoire ou sur une défaite et à afficher un message en conséquence.

Il reste cependant un cas que nous n'avons pas traité : le joueur pourrait proposer plusieurs fois la même lettre. Dans ce cas, le jeu ne devrait pas tenir compte des lettres déjà proposées. En fait, ceci est assez simple à ajouter à notre programme. Une possibilité (il y en a d'autres) est la suivante :

- Nous allons utiliser une troisième chaîne de caractères contenant les lettres incorrectes déjà proposées. Au départ, cette chaîne est vide (""). A chaque fois que le joueur propose une lettre incorrecte (cas « si non ... » ci-dessus), on va non seulement augmenter le compteur d'erreurs, mais aussi ajouter la lettre aux lettres incorrectes.
- Nous allons également ajouter une étape dans la boucle : après que le joueur ait proposé une lettre, mais avant de vérifier si elle est dans le mot à trouver, nous allons vérifier si la proposition est dans la liste de lettres incorrectes (notez que nous pouvons le faire très simplement en réutilisant la fonction `EstDansMot()`). Si non, on poursuit le traitement comme ci-dessus (tester si la lettre est dans le mot à trouver...) ; si oui, on ne fait rien (on ignore donc la lettre).
- Et que faire si l'utilisateur propose plusieurs fois la même lettre *correcte* ? Nous avons plusieurs possibilités : on pourrait ajouter une chaîne de caractères contenant les lettres correctes déjà proposées, et faire un test supplémentaire. On pourrait aussi stocker dans la liste toutes les lettres, correctes ou non, déjà proposées (il faut alors changer l'endroit où l'on ajoute les nouvelles lettres à la liste). Une troisième possibilité est de... ne rien faire ! De la manière dont la fonction a été écrite ci-dessus, cela fonctionnera : nous n'utilisons pas de compteur de bonnes réponses, juste un compteur d'erreurs. Si une lettre correcte est proposée plusieurs fois, le programme va constater plusieurs fois qu'elle est dans le mot et ajouter plusieurs fois la lettre à sa place dans le mot à trouver, mais cela ne va rien changer. Cela ajoute certes quelques opérations inutiles, mais pas assez pour que ce soit vraiment gênant².
- Il ne reste plus qu'à modifier la fonction `AfficherSituation()` pour qu'elle affiche aussi la liste des lettres incorrectes déjà proposée.

² Même si cela mérite débat: si nous voulions vraiment optimiser les performances, la question serait de savoir si le temps perdu à refaire un traitement inutile compense le temps passé à faire un test supplémentaire. En fait, la réponse dépend du nombre moyen de fois où un joueur proposerait plusieurs fois la même lettre.

Partie 4 : interface graphique

Pour parfaire le jeu, nous allons améliorer l'affichage pour dessiner un pendu.

L'objectif est d'obtenir un dessin semblable à la figure ci-contre. Comme nous le voyons, il est principalement constitué de lignes verticales, horizontales et diagonales. C'est donc certainement une bonne idée d'écrire des fonctions génériques dessinant ces différents types de lignes.



- 12. Ecrire une fonction `LigneHorizontale()`. Cette fonction reçoit 3 entiers correspondant à la position horizontale et verticale du début de la ligne, ainsi qu'à sa longueur, et dessine la ligne correspondante à l'écran.**

Indices :

- La fonction `Console.SetCursorPosition()` permet de placer le curseur à l'endroit voulu sur la console.

- 13. Ecrire une fonction `LigneDiagonaleGauche()`. Cette fonction reçoit 3 entiers correspondant à la position horizontale et verticale du début de la ligne, ainsi qu'à sa longueur, et dessine à l'écran une ligne diagonale descendant vers la gauche.**

- 14. Ecrire une fonction `LigneDiagonaleDroite()`. Cette fonction reçoit 3 entiers correspondant à la position horizontale et verticale du début de la ligne, ainsi qu'à sa longueur, et dessine à l'écran une ligne diagonale descendant vers la droite.**

Il nous faut aussi une fonction assez simple qui dessine la tête du pendu.

- 15. Ecrire une fonction `DessinerTete()`. Cette fonction reçoit 2 entiers correspondant à la position horizontale et verticale du coin supérieur gauche, et dessine la tête du pendu.**

Et nous pouvons enfin finaliser la partie graphique :

- 16. Modifier la fonction `AfficherPendu()`. L'interface (paramètres reçus, nom,...) reste la même, mais la fonction affiche un dessin partiel du pendu en fonction du nombre d'erreurs commises.**

Indices :

- La fonction dessine de toute façon la potence et la corde.
- Ensuite, s'il y a au moins une erreur, il faut dessiner la tête ; s'il y en a au moins 2, il faut dessiner le corps, et ainsi de suite.

- Pour les différents dessins, il suffit d'employer nos fonctions dessinant des lignes, et de tâtonner un peu au besoin.

Voilà ! Le jeu est enfin complet. Pour le peaufiner, il peut être intéressant de revoir la fonction `AfficherSituation()` pour qu'elle affiche les informations à des endroits précis. Pour éviter des caractères parasites, on peut aussi nettoyer l'écran avant chaque affichage, grâce à la fonction `Console.Clear()`.